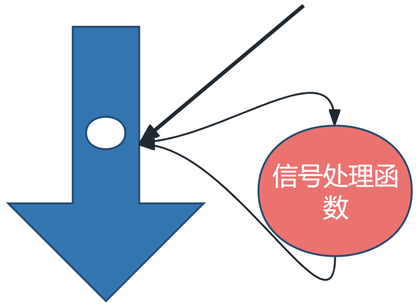

内核

信号(signal):软中断，异步通信模式
信号的发送:
1.kill命令
2.kill函数
3.用户交互操作(快捷键/鼠标动作)
3.sigqueue发送携带数据的信号

```
hgg@hgg-625:~$ kill  -1  6454
```

```
hgg@hgg-625:~$ kill  -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

信号(signal)的分类:
实时信号（可靠信号33-64）：按顺序确保处理
非实时信号（不可靠信号1-31）:不保证处理顺序，也不保证一定处理

信号的处理:
1.默认行为
2.忽略,并不是所有的信号都可以被忽略(比如9)
3.收到信号后执行信号处理函数，然后再回到原流程
4.暂时阻塞，延迟处理
#kill发信号，signal能处理
#siqueue发信号，如果携带数据，必须sigaction;
如果不携带信号，sigaction和signal都能处理
sigaction应用时，调整struct sigaction 的sa_flags
设置为SA_SIGINFO,信号的处理函数切换到sa_sigaction

信号处理

```
1  /* signal handlers */
2          struct signal_struct *signal;
3          struct sighand_struct *sighand;
4
5          sigset_t blocked, real_blocked;
6          sigset_t saved_sigmask; /* restored if set_restore_sigmask() was used */
7          struct sigpending pending;
8
9          unsigned long sas_ss_sp;
10         size_t sas_ss_size;
11         int (*notifier)(void *priv);
12         void *notifier_data;
13         sigset_t *notifier_mask;
```

signal 指向信号描述符
sighand 指向信号处理程序描述符
blocked 表示被阻塞信号的掩码
real_blocked 表示临时掩码
pending 中存放未决信号(未被处理的信号)
sas_ss_sp 是信号处理程序备用堆栈的地址
sas_ss_size 表示堆栈的大小
设备驱动程序常用notifier指向的函数来阻塞进程的某些信号。（notifier_data是可能用到的数据，notifier_mask是信号的位掩码）

```
int sigaction(int signum,
              const struct sigaction *_Nullable restrict act,
              struct sigaction *_Nullable restrict oldact);

struct sigaction {
              void     (*sa_handler)(int);
              void     (*sa_sigaction)(int, siginfo_t *, void *);
              sigset_t   sa_mask;
              int        sa_flags;
              void     (*sa_restorer)(void);
          };
```